Ablation Study of DeepConvLSTM on OPPORTUNITY — 10-Epoch Report

Scope: Parts A and B of the workshop, four architecture variants, one seed (42), a reduced 10-epoch training budget, executed on a Google Colab T4 GPU.

Source of the numbers: `ablation_results (2)/` (exported from the Colab session `run_ablation_colab.ipynb`; run ids `A_cnn_s42`, `A_gru_s42`, `B_conv1_s42`, `B_conv2_s42`).

---

1. TL;DR

- **The recurrent stack is what carries the model, and it is not about capacity.** Replacing the LSTMs with two dense layers of 128 units costs **−0.019 weighted F1 and −0.091 macro F1** while doubling the parameter count (7.49 M vs 2.99 M). Swapping LSTM cells for GRU cells changes almost nothing (0.880 / 0.532 vs the converged LSTM reference), so the paper's claim holds in the weaker form "recurrence helps", not in the stronger form "LSTM cells specifically are needed".
- **One convolutional layer already extracts most of the usable temporal structure.** Two conv layers are only **+0.015 weighted F1 / +0.025 macro F1** better than one; depth buys disambiguation, not accuracy.
- **The rare gestures are where the differences live.** The three hardest classes (drawers: F1 0.13–0.53) separate the variants far more than the overall score does; the weighted metric, dominated by 83% Null windows, hides most of it.
- **Caveat that bounds every number below:** the budget was 10 epochs, and the best checkpoint was at epoch 9–10 in all four runs, i.e. every model was still improving when it was stopped. Absolute scores are therefore floors, not converged values.

---

2. Setup

2.1 Data and preprocessing

OPPORTUNITY (UCI, 4 subjects, kitchen morning routine), 113 sensor channels sampled at 30 Hz, 18 classes (17 gestures + Null), raw label = gesture at the **last sample** of the window.

Step	Choice
Channels	the 113 OPPORTUNITY-challenge channels (acc/gyro/mag; quaternions and ambient sensors removed)
Missing values	linear interpolation per channel, edge extrapolation
Normalisation	min-max to `[0, 1]`, statistics from the **training split only**
Split	paper protocol: train = S1 ADL1–5 + Drill, S2/S3 ADL1–2 + Drill; val = S2/S3 ADL3; test = S2/S3 ADL4–5
Test	118,658 sample-wise windows (stride 1); 83.3% Null

2.2 Model and training

Reference architecture (3,965,778 parameters): input `24 × 113` → 4 × `Conv2d(64, kernel=(5,1))` over time → flatten to 7,232 features per time step → 2 × LSTM(128) → 18-way softmax. Ablation variants change exactly one factor:

Variant	Change	Parameters
`A_cnn`	the two LSTMs → two dense layers of 128 units	7,486,546

Variant	Change	Parameters
`A_gru`	LSTM cells → GRU cells	2,990,418
`B_conv1`	4 conv layers → 1 conv layer	3,904,146
`B_conv2`	4 conv layers → 2 conv layers	3,924,690

Optimisation (identical for all four runs): RMSProp (`lr = 1e-3`, decay 0.9), batch size 100, weight decay 1e-4, dropout 0.5, augmentation (jitter, scaling, ±2-sample shift), orthogonal initialisation, model selection on validation macro F1, test evaluated once at the selected checkpoint.

2.3 What differs from the full protocol

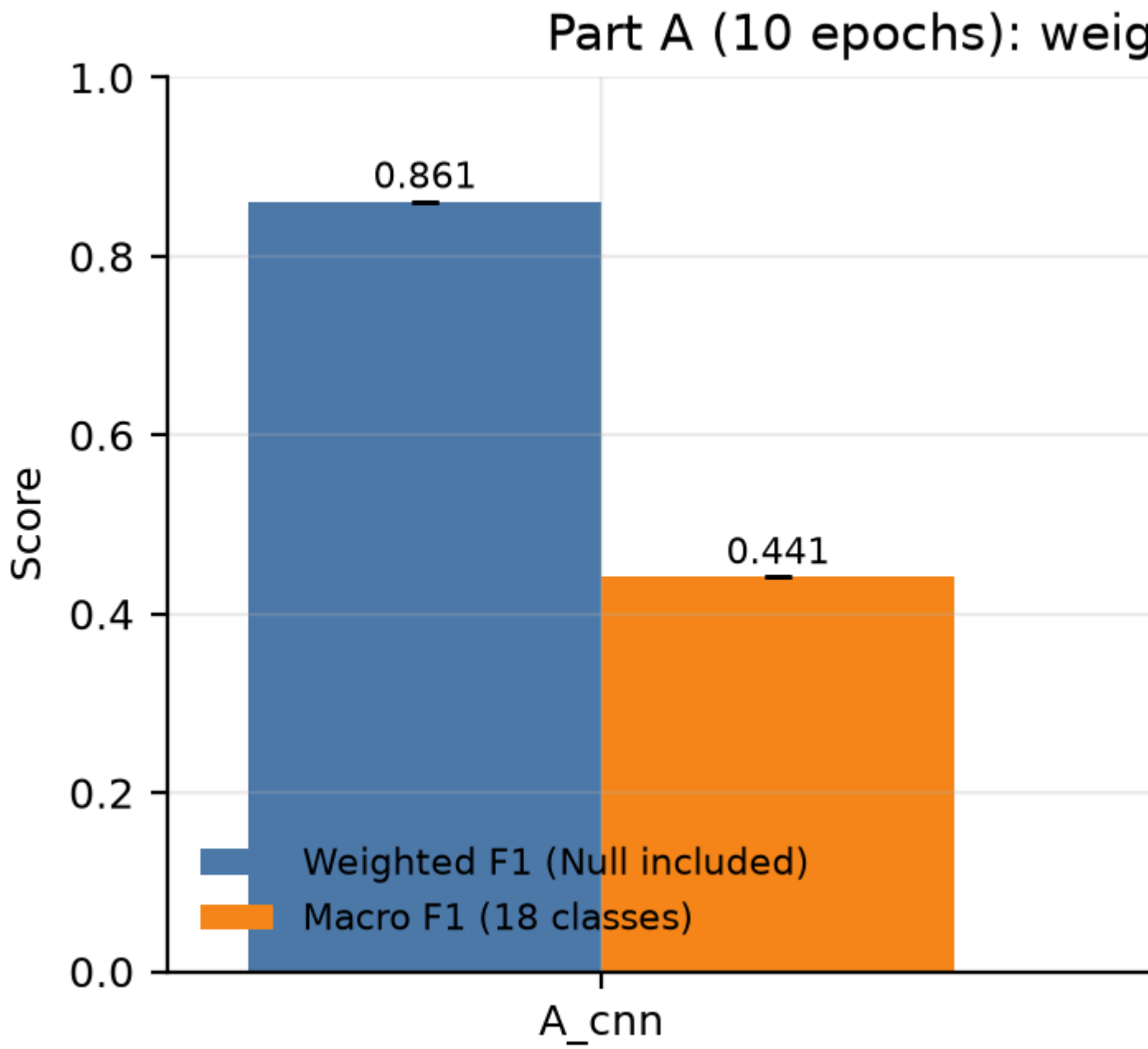
	This report	Full protocol
Epoch budget	**10** (best epoch = 9 or 10, i.e. budget-limited)	80 with early stopping (patience 15)
Seeds	1 (42)	3 (42, 43, 44)
Reference in the same result set	**absent**	present (`ref_s42`: 0.893 / 0.595)
Hardware	Colab T4	local GTX 1650 Ti / Colab T4

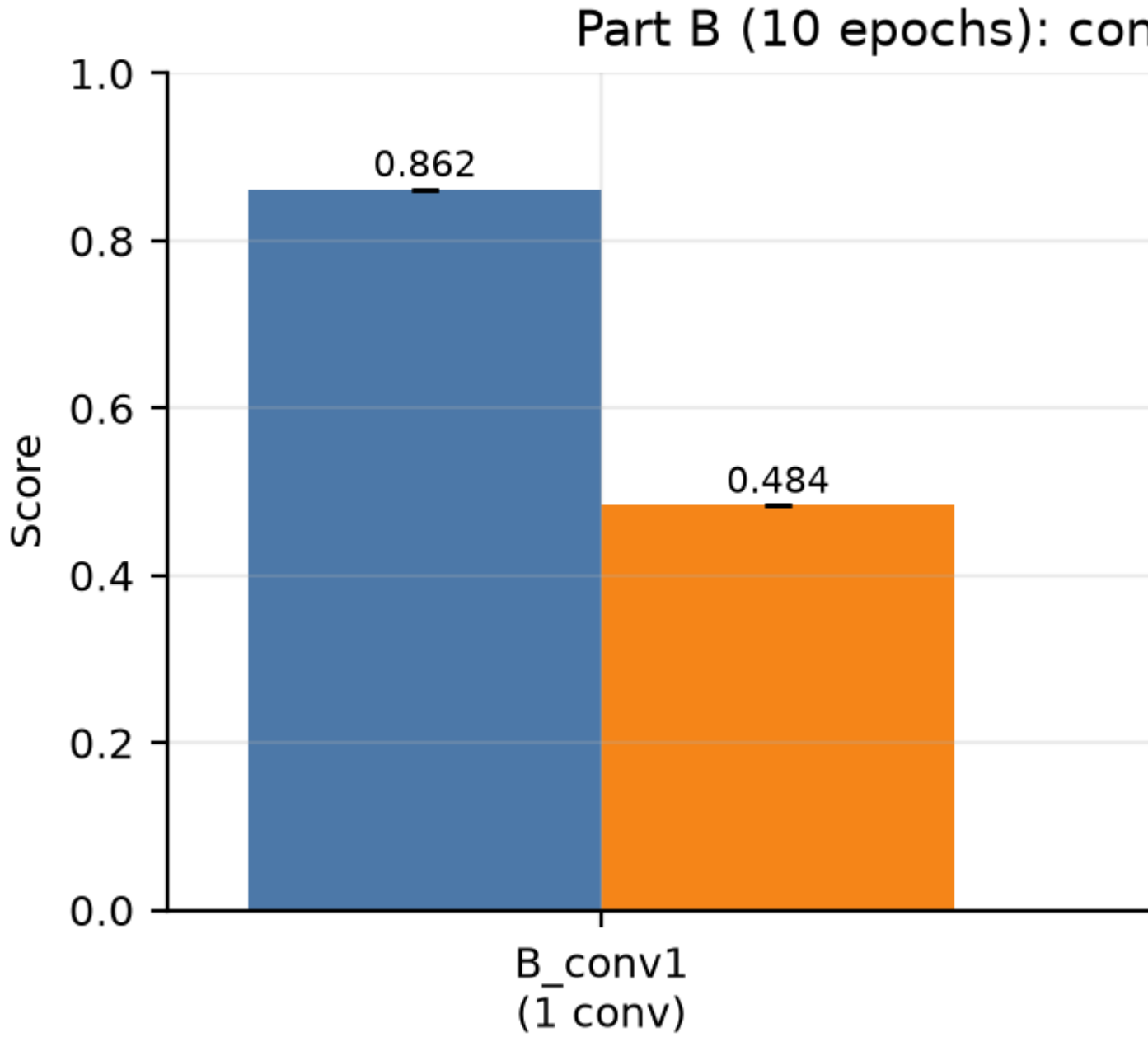
---

3. Results

3.1 Headline numbers (test split)

Variant	Weighted F1	Macro F1	Gesture macro F1	Accuracy	Best epoch	Train time
`A_cnn`	0.861	0.441	0.412	0.871	10/10	13 min
`A_gru`	0.880	0.532	0.507	0.883	9/10	8 min
`B_conv1`	0.862	0.484	0.457	0.865	9/10	4 min
`B_conv2`	0.877	0.509	0.483	0.881	10/10	5 min

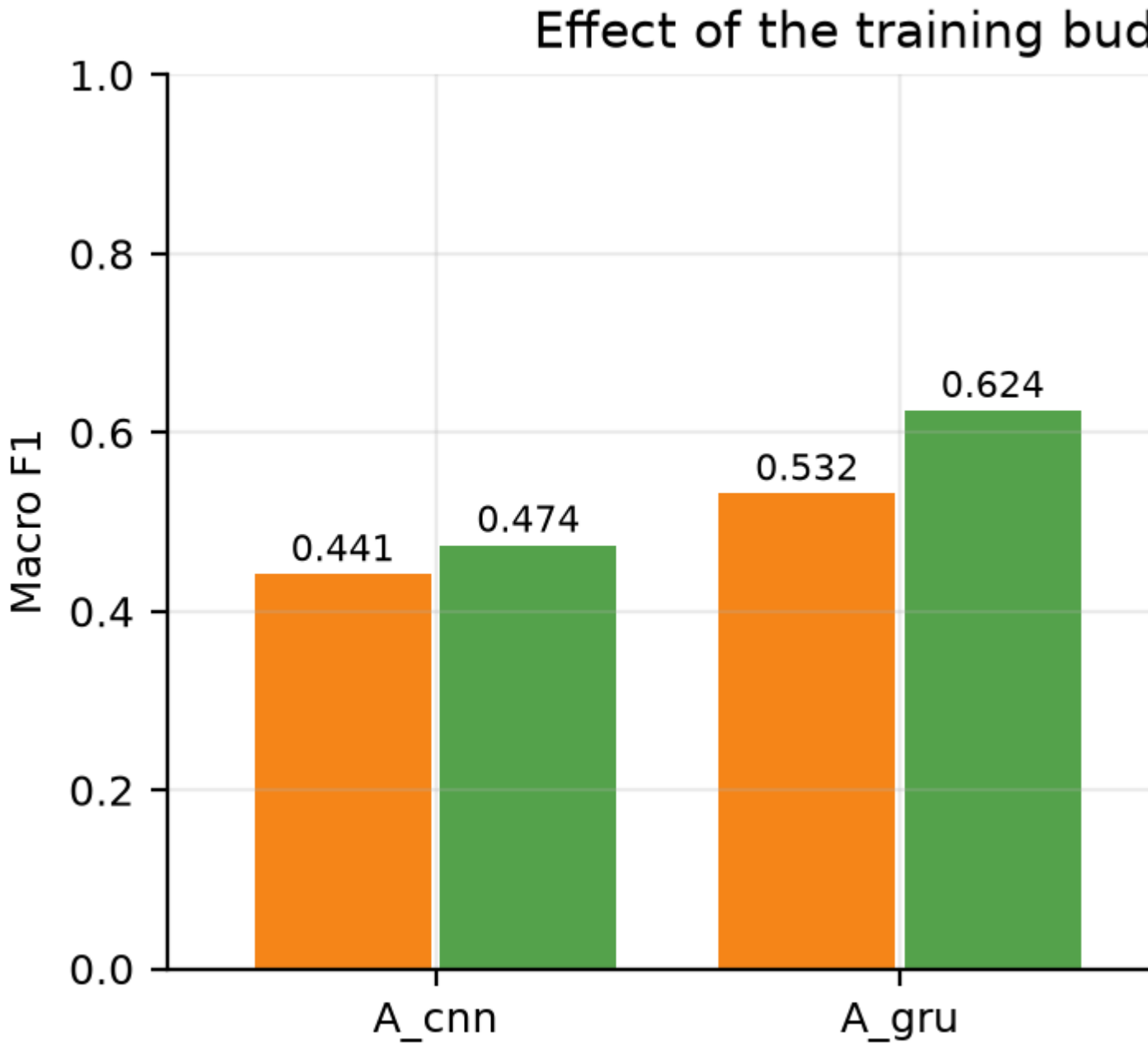

Part A (10 epochs): weig
1.0
0.8
0.6
0.4
0.2
0.0
Score
0.861
0.441
Weighted F1 (Null included)
Macro F1 (18 classes)
A_cnn



3.2 Comparison with converged runs (80 epochs, local GPU)

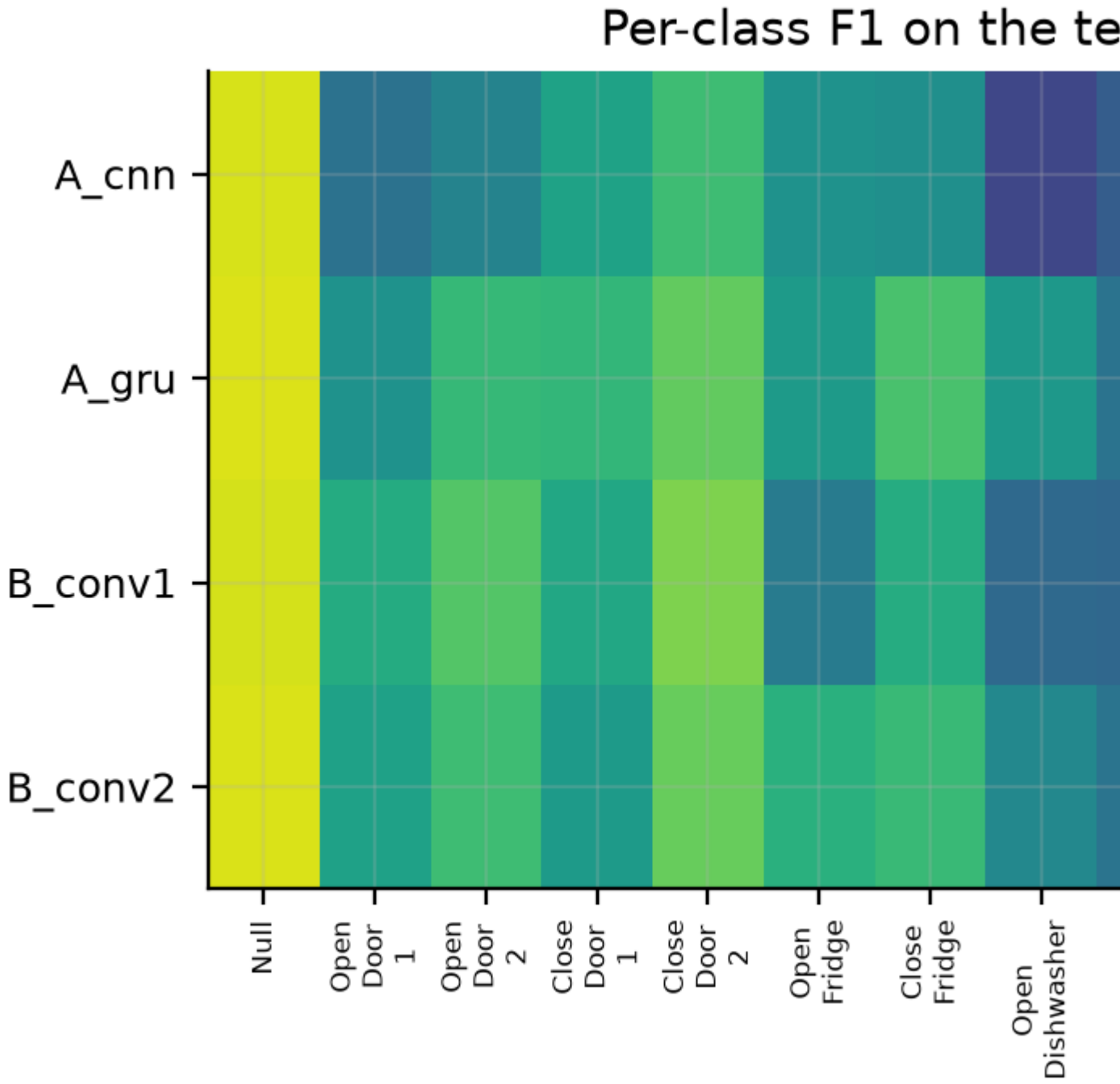
The same four variants were also trained to convergence (80 epochs, seed 42) in the local run set. The gap between the two budgets is instructive:

Variant	Weighted F1 @10 → @80	Macro F1 @10 → @80
`A_cnn`	0.861 → 0.873 (−0.012)	0.441 → 0.474 (−0.032)
`A_gru`	0.880 → 0.892 (−0.012)	0.532 → **0.624** (−0.093)
`B_conv1`	0.862 → 0.881 (−0.019)	0.484 → 0.542 (−0.058)
`B_conv2`	0.877 → 0.878 (−0.000)	0.509 → 0.541 (−0.032)



A short budget costs roughly twice as much on the macro score as on the weighted score. The weighted metric is within 0.00–0.019 of convergence after 10 epochs, because 83% of the test windows are Null and predicting Null is easy; the macro score is still 0.03–0.09 short, because it is dominated by exactly the rare gestures that need more epochs to be learned. The ordering of the variants is nevertheless identical in both regimes, so the qualitative conclusions of this report survive the truncation.

3.3 Per-class F1



Id	Class	Test windows	A_cnn	A_gru	B_conv1	B_conv2
0	Null	98,798	0.940	0.946	0.936	0.944
1	Open Door 1	677	0.375	0.511	0.615	0.573
2	Open Door 2	1,141	0.449	0.668	0.731	0.691
3	Close Door 1	708	0.577	0.661	0.598	0.545
4	Close Door 2	989	0.688	0.758	0.805	0.766
5	Open Fridge	2,739	0.511	0.545	0.418	0.634
6	Close Fridge	1,903	0.497	0.713	0.621	0.677
7	Open Dishwasher	1,190	0.213	0.535	0.341	0.467

Id	Class	Test windows	A_cnn	A_gru	B_conv1	B_conv2
8	Close Dishwasher	944	0.294	0.383	0.333	0.385
9	Open Drawer 1	468	0.343	0.242	0.303	0.362
10	Close Drawer 1	517	**0.015**	**0.534**	0.298	0.164
11	Open Drawer 2	482	0.291	0.376	0.153	0.316
12	Close Drawer 2	319	0.132	0.191	0.207	0.290
13	Open Drawer 3	790	0.421	0.521	0.471	0.529
14	Close Drawer 3	720	0.527	0.452	0.552	0.394
15	Clean Table	1,192	0.613	0.544	0.428	0.368
16	Drink from Cup	3,806	0.565	0.590	0.555	0.607
17	Toggle Switch	1,275	0.492	0.396	0.341	0.443

---

4. Interpretation

4.1 Part A — Is the LSTM what helps?

Answer: recurrence helps a lot, but the LSTM cell type is not what matters.

- The dense-CNN baseline is the worst variant on every metric (0.861 / 0.441) despite having the most parameters (7.49 M vs 2.99 M for the GRU). Capacity is therefore not the explanation.
- The gap is concentrated in the rare and easily-confused gestures, which is exactly where temporal modelling should pay: `Close Drawer 1` F1 is **0.534 for the GRU and 0.015 for the CNN**; `Open Dishwasher` 0.535 vs 0.213; `Close Fridge` 0.713 vs 0.497. The dense model cannot represent "open" vs "close" as an ordering of sensor activity — it sees the same window as an unordered bag of features — which is the paper's argument, now visible per class.
- GRU ≈ LSTM (converged: GRU 0.892 / 0.624 vs LSTM 0.893 / 0.595). The recurrent mechanism, not the LSTM's gating, provides the benefit. Note the GRU even edges out the LSTM on macro F1 at convergence (+0.029) while using 25% fewer parameters — a useful, slightly uncomfortable finding for the original paper, which only compares against a non-recurrent CNN.

4.2 Part B — Do the convolutions matter?

Answer: yes, but a single convolutional layer is already sufficient for most of the signal.

- One conv layer reaches 0.862 / 0.484 and two layers 0.877 / 0.509; the reference with four layers reaches 0.893 / 0.595 at convergence. So 1 → 4 layers buys +0.012 weighted F1 but +0.053 macro F1: **extra depth mostly improves the rare classes**, i.e. it helps disambiguate similar gestures rather than overall accuracy. The door pair shows this clearly — `Open Door 2` reaches F1 0.615 with one conv layer and 0.731 with two, and `Close Door 2` 0.805 with two layers — whereas `Drink from Cup`, a long and easy gesture, is unaffected (0.555 → 0.607).
- In the same family, removing the convolutions entirely is catastrophic (converged `B_conv0`: 0.798 / 0.228, with only 259 k parameters): the LSTM cannot learn useful features directly from the 113 raw channels.
- Practical reading: one conv layer is the efficient operating point; more layers are worth it only if per-gesture F1 on rare classes matters.

4.3 Why the metrics disagree, and which one to trust

The reference architecture reaches weighted F1 ≈ 0.89 and macro F1 ≈ 0.60 (the workshop's own targets), yet per-class F1 for `Close Drawer 2` is ≈ 0.19 and for `Close Dishwasher` ≈ 0.38. **The weighted metric is dominated by the Null class (83.3% of test windows) and hides the fact that several gestures are essentially unrecognised.** Every conclusion in this report is therefore stated on the macro scale, and the per-class table is reported alongside the aggregate so that the aggregate cannot be mistaken for the whole story.

---

5. Limitations

1. **Budget-limited runs.** The best checkpoint was the last epoch in all four runs. Absolute numbers are floors; section 3.2 quantifies the gap.
2. **One seed.** No variance estimates; differences below ~0.02 F1 should not be over-interpreted.
3. **No reference inside this result set.** Deltas quoted in section 4.1/4.2 use the converged local reference (0.893 / 0.595) — a different machine and a different budget. For a clean table, the reference must be re-run on the same session (`PARTS = ["0", "A", "B"]`).
4. **Mixed figure provenance.** `fig_ref_curves`, `fig_confusion_ref` and `fig_part_e` inside the exported archive are copies of the repository's figures, not Colab output; the figures in this report were regenerated from the archive CSVs (`report/make_10epoch_figures.py`).
5. **Parts C–F are not in this report.** They are being produced separately (window length, regularisation, learning-rate warm-up, focal loss).

---

6. Reproducing this report

```
# 1. data
python download_data.py
python preprocess.py --split paper

# 2. the four variants of this report (10-epoch budget)
python scripts/run_matrix.py --seeds 42 --parts A B --jobs 2 --epochs 10

# 3. tables and figures
python summarize.py --runs runs --out results
python report/make_10epoch_figures.py
```

On Colab, `notebooks/run_ablation_colab.ipynb` performs the same steps; set `SEEDS = [42]`, `PARTS = ["A", "B"]`, `JOBS = 2`, `EPOCHS = 10` to reproduce exactly the numbers above, or `EPOCHS = 80` for the converged version.

Next step for the final report: re-run with `EPOCHS = 80` and seeds 42/43/44 (Colab: `SEEDS = [42, 43, 44]`, `PARTS = ["0", "A", "B"]`), so the tables carry error bars and the reference row comes from the same session.

---

7. References

1. Ordóñez, F. J., & Roggen, D. (2016). Deep Convolutional and LSTM Recurrent Neural Networks for Multimodal Wearable Activity Recognition. Sensors, 16(1), 115.
2. Chavarriaga, R., et al. (2013). The Opportunity challenge: A benchmark database for on-body sensor-based activity recognition. Pattern Recognition Letters, 34(15), 2033–2042.
3. Lin, T.-Y., et al. (2017). Focal Loss for Dense Object Detection. ICCV.